Self-Learning Plan: React Native + Expo in 20 Days

20 study days inside 30 calendar days · 4 weeks · 4 tasks per week · self-paced, no teacher.

Every task adds a small, real feature to this app. Before each task you learn exactly the concepts it needs. Level: **easy to medium**.

How it works

The rule for every day: learn first, then do the task

Part	Time	What you do
1. Learn	1–1.5 hours	Read the "What to learn" links of the day's task. Understand the idea.
2. Do the task	2–2.5 hours	Build it in the app, using what you just learned and the code it points to.
3. Check & share	30 minutes	`npm run validate`, test on your phone, push, open a PR `Task 1.2: …`.

The week

Day of the week	What happens
Monday	Learn + **Task 1** of the week
Tuesday	Learn + **Task 2**
Wednesday	Learn + **Task 3**
Thursday	Learn + **Task 4**
Friday	**Catch-up and weekly check**: finish any task, do the weekly checklist
Saturday	Optional: bonus task, or rest
Sunday	Rest

The 30 days

Week	Calendar days	Theme	The 4 tasks
1	1–7	Screens and UI	Support row · Rating stars · Details card · Reviews screen
2	8–14	Data: API, React Query, forms	Category screen · Sort · Out-of-stock action · Discount field
3	15–21	Build your own module	Users data · Users list · User profile · Login
4	22–28	Polish and mini project	Offline banner · Home widget · Cart tab · Cart summary
–	29–30	Look back	Demo recording + self-check

When you're stuck

1. Re-read the task and open the file it points to: the pattern usually already exists in the app.
2. Copy the **exact** error message and search for it.
3. Read the "What to learn" links again.
4. If you use an AI assistant, ask it to **explain**, not to write the whole thing.
5. Still stuck after 45 minutes? Write the blocker in your PR description and move on.

Setup (before Week 1)

- [ ] Install **Node.js** (LTS), **Git**, **VS Code** (ESLint, Prettier, Tailwind CSS IntelliSense extensions) and **Expo Go** on your phone (SDK 58).
- [ ] Create a **GitHub** account and **fork** this repository (how to fork).
- [ ] Clone your fork, `npm install`, `npm start`, scan the QR code. Full steps: GETTING-STARTED.md.
- [ ] Create `PROGRESS.md` in your fork (template at the end) and update it after every task.
- [ ] Know how to open a PR from your fork (guide).

Cheat sheet: building screens in this app (keep it open)

You don't design from scratch. You build with pieces the app already has. See them all in the app: **Settings → UI Sandbox**.

You need	Use
Text	`Text` from `@/shared/ui/text` (`variant="h3"`, `"large"`, `"muted"`)

You need	Use
Button	`Button` from `@/shared/ui/button` (`variant="outline"`, `"destructive"`)
Card / box	`className="rounded-lg border border-border bg-card p-4"`
Badge, Avatar, Input	`@/shared/ui/badge` · `@/shared/ui/avatar` · `@/shared/ui/input`
Icon	`Icon` from `@/shared/ui/icon` + an icon from `lucide-react-native`
Loading/error/empty	`LoadingState`, `ErrorState`, `EmptyState` from `@/shared/components/screen-states`

Colors: never hex (`#4f46e5`). Use theme classes so dark mode works by itself: `bg-background` · `bg-card` · `text-foreground` · `text-muted-foreground` · `text-primary` · `bg-primary` · `text-destructive` · `bg-success` · `border-border`.

Building…	Copy from
A list screen	`src/modules/catalog/features/product-list/product-list-screen.tsx`
A details screen	`src/modules/catalog/features/product-detail/product-detail-screen.tsx`
A form	`src/modules/catalog/components/product-form.tsx`
A settings-style list	`src/modules/settings/features/app-settings/settings-screen.tsx`

Design inspiration (optional): the free Obra shadcn/ui Figma kit uses the same components as this app.

Week 1: Screens and UI (calendar days 1–7)

Day	Learn	Task
Monday	The project, React Native components	Task 1.1 Support row
Tuesday	React components and props, TypeScript	Task 1.2 Rating stars
Wednesday	Flexbox, theme colors, zod basics	Task 1.3 Details card
Thursday	Expo Router: routes and navigation	Task 1.4 Reviews screen
Friday	–	Catch-up + weekly check

Task 1.1 — "Contact support" row in Settings

When: Monday (Day 1). Difficulty: easy.

Concepts you need: how the project is organised · React Native components · opening other apps with `Linking`.

What to learn first:

- README: "What the app does" and "Project structure"
- React Native: Linking

The task:

1. Open `src/modules/settings/features/app-settings/settings-screen.tsx`.
2. Copy the "UI Sandbox" `SettingsRow`. Change it to a `Mail` icon and the label "Contact support".
3. On press, call `Linking.openURL('mailto:support@example.com')`.

Done when:

- [ ] Tapping the row opens the email app on your phone.
- [ ] Your first PR `Task 1.1: contact support row` is open.

Task 1.2 — `<Rating />` stars component

When: Tuesday (Day 2). Difficulty: easy–medium.

Concepts you need: components and props · typing props with TypeScript · rendering a list with `map` and `key`.

What to learn first:

- React: Quick Start
- TypeScript for React: Basic

The task:

1. Create `src/modules/catalog/components/rating.tsx` with props `value: number` and `showValue?: boolean`.
2. Draw 5 stars: filled up to the rounded value, empty after. See how `product-detail-screen.tsx` draws one star with `<Icon as={Star} … />`.
3. Use `<Rating />` in `src/modules/catalog/features/product-list/product-card.tsx` instead of the current star and number.

Done when:

- [ ] Product cards show 5 stars, in light and dark mode.

Task 1.3 — "Details" card on the product screen

When: Wednesday (Day 3). Difficulty: medium.

Concepts you need: Flexbox (rows and columns) · theme color classes · adding fields to a zod schema · conditional rendering.

What to learn first:

- React Native: Flexbox
- THEMING.md: "The tokens"
- Zod: "Basic usage"

The task:

1. Open `https://dummyjson.com/products/1` in your browser. Find `warrantyInformation`, `shippingInformation` and `returnPolicy`.
2. Add them to `productSchema` (`src/modules/catalog/data/product.schema.ts`) as `z.string().optional()`.
3. In `product-detail-screen.tsx`, under the description, add a card with three rows: label on the left (`text-muted-foreground`), value on the right. Hide a row when its value is missing.

Done when:

- [ ] The card shows on product 1, in light and dark mode. `npm run validate` passes.

Task 1.4 — Reviews screen

When: Thursday (Day 4). Difficulty: medium.

Concepts you need: file-based routes · dynamic routes (`[id]`) · navigating with `router.push` · a feature folder with an `index.ts` .

What to learn first:

- Expo Router: introduction
- ARCHITECTURE.md: "Navigation"
- Feature reference: "What is a feature?"

The task:

1. Add a `reviewSchema` (`rating` , `comment` , `reviewerName`) and `reviews: z.array(reviewSchema).default([])` to `productSchema` .
2. Create `src/modules/catalog/features/product-reviews/` with `product-reviews-screen.tsx` and `index.ts` (copy the shape of `features/product-detail/`). Export the screen from `src/modules/catalog/index.ts` .
3. Add the route `src/app/products/[id]/reviews.tsx` (copy `src/app/products/[id]/edit.tsx`) and register it in `src/app/_layout.tsx` with the title "Reviews".
4. Add a "See all reviews" button on the product screen that opens it.

Done when:

- [ ] You can open any product's reviews and go back.

Friday — Week 1 check

- [ ] All 4 PRs open; `PROGRESS.md` updated.
- [ ] Every screen you touched looks right in dark mode (Settings → Theme).
- [ ] **Bonus (optional):** a "Share" button on the product screen with React Native Share.

Week 2: Data — API, React Query and forms (calendar days 8–14)

Day	Learn	Task
Monday	How data flows · React Query basics	Task 2.1 Category screen
Tuesday	Query keys	Task 2.2 Sort by price
Wednesday	Mutations (changing data)	Task 2.3 Out-of-stock action
Thursday	Forms and validation	Task 2.4 Discount field
Friday	–	Catch-up + weekly check

Task 2.1 — "Products by category" screen

When: Monday (Day 6). Difficulty: medium.

Concepts you need: the layers (screen → hook → service → HTTP client) · endpoints · a service function · `useQuery` · loading / error / empty states.

What to learn first:

- ARCHITECTURE.md: "The layers inside a slice" and "READ: the list"
- TanStack Query overview

The task:

1. Follow ADDING-A-FEATURE.md step by step. It builds exactly this screen and includes the code.
2. On the product detail screen, make the category badge tappable so it opens the new screen.

Done when:

- [ ] Tapping "smartphones" on a phone's detail page lists all smartphones.

Task 2.2 — "Sort by price" button

When: Tuesday (Day 7). Difficulty: medium.

Concepts you need: query keys (every input that changes the result must be in the key) · passing parameters from a screen to a hook to a service.

What to learn first:

- TkDodo: Effective React Query Keys

- Read `src/modules/catalog/data/product.queries.ts` and `product.keys.ts`

The task:

1. Add a button above the product list that cycles: Default → Price low to high → Price high to low.
2. `productsService.list` already accepts `sortBy` and `order`. Pass them through `useProductList` and `productQueries.list`.
3. Add them to the key: `productKeys.list({ search, sortBy, order })`.

Done when:

- [ ] The order changes when you tap the button, and search still works.

Task 2.3 — "Mark out of stock" action

When: Wednesday (Day 8). Difficulty: medium.

Concepts you need: mutations (`useMutation`) · updating the cache after a change · reusing a hook from another feature through its `index.ts`.

What to learn first:

- TkDodo: Mastering Mutations
- Read `src/modules/catalog/features/edit-product/use-update-product.ts`

The task:

1. In `product-actions-sheet.tsx` (opens with a long-press on a product), add a "Mark out of stock" item.
2. On tap, call `useUpdateProduct()` with `stock: 0`. Payload hint: `productFormSchema.parse({ ...productToFormInput(product), stock: '0' })`.

Done when:

- [ ] The stock badge says "Out of stock" right away, in the list and on the detail screen.

Task 2.4 — "Discount %" field in the product form

When: Thursday (Day 9). Difficulty: medium.

Concepts you need: react-hook-form · validation rules with zod · writing a schema test.

What to learn first:

- React Hook Form: get started
- Read `src/modules/catalog/components/product-form.tsx` and `data/__tests__/product.schema.test.ts`

The task:

1. Add `discountPercentage` to `productFormSchema`: optional, a number from 0 to 90.
2. Add the input to the form (copy the "Stock" field) with `keyboardType="number-pad"`.
3. Add a test that rejects `95`.

Done when:

- [ ] Typing `95` shows an error; `20` saves and shows the discount on the product.

Friday — Week 2 check

- [ ] All 4 PRs open; `PROGRESS.md` updated; `npm test` passes.
- [ ] **Bonus (optional):** a test in `products.service.test.ts` that a 404 throws an `ApiError` with `isNotFound === true`.

Week 3: Build your own module (calendar days 15–21)

This week you build a **Users** section from zero with ADDING-A-MODULE.md, which has the code for every step.

Day	Learn	Task
Monday	What a module is · writing tests	Task 3.1 Users data layer
Tuesday	Lists with FlashList · adding a tab	Task 3.2 Users list
Wednesday	Dynamic routes · Linking	Task 3.3 User profile
Thursday	Login with tokens · SecureStore	Task 3.4 Login
Friday	–	Catch-up + weekly check

Task 3.1 — Users data layer (with tests)

When: Monday (Day 11). Difficulty: medium (the recipe has the code).

Concepts you need: a module and its public API (`index.ts`) · endpoints, schema, service, query keys, queries · unit tests with a fake `fetch` .

What to learn first:

- Module reference
- ADDING-A-MODULE.md: steps 1–5
- Jest: getting started · DummyJSON users

The task:

1. Create `src/modules/users/` with endpoints, schema, service, query keys and queries.
2. Copy `products.service.test.ts` to `src/modules/users/data/__tests__/users.service.test.ts` and write 3 tests: the list loads, a user loads by id, a 404 throws.

Done when:

- [ ] `npm run typecheck` and `npm test` pass.

Task 3.2 — "Users" tab

When: Tuesday (Day 12). Difficulty: medium.

Concepts you need: FlashList · a feature screen · a tab route.

What to learn first:

- FlashList: "Usage"
- ADDING-A-MODULE.md: steps 6–9

The task:

1. Feature `src/modules/users/features/user-list/`: one row per user with an `Avatar`, the full name and the email (`text-muted-foreground`). Copy `product-list-screen.tsx` and simplify it (no search).
2. Route `src/app/(tabs)/users.tsx` and a tab with the `Users` icon in `src/app/(tabs)/_layout.tsx`.

Done when:

- [ ] A Users tab shows a scrolling list with photos, in light and dark mode.

Task 3.3 — User profile screen

When: Wednesday (Day 13). Difficulty: medium.

Concepts you need: dynamic routes with params · reading `id` in the route file and passing it to the screen · `Linking` for `tel:` and `mailto:`.

What to learn first:

- Expo Router: introduction (dynamic routes)
- Read `src/app/products/[id]/index.tsx` (how a route passes the id to a screen)

The task:

1. Feature `features/user-detail/`, route `src/app/users/[id].tsx`.
2. Show a big avatar, name, email, phone and city, plus **Call** and **Email** buttons.
3. Tapping a user in the list opens their profile.

Done when:

- [ ] The Call button opens the phone dialer.

Task 3.4 — Login screen

When: Thursday (Day 14). Difficulty: medium.

Concepts you need: a form that calls an API (`POST`) · access tokens · storing secrets with SecureStore.

What to learn first:

- DummyJSON auth
- Expo SecureStore

The task:

1. Create a small `auth` module with a login screen (username + password; copy the form pattern from `product-form.tsx`) that calls `POST /auth/login`. Test user: `emilys` / `emilyspass`.
2. Save the `accessToken` with SecureStore (`npx expo install expo-secure-store`).
3. In Settings, show "Signed in as Emily" and a "Log out" button that deletes the token. Open the login screen from Settings.

Done when:

- [ ] Close and reopen the app: still signed in. Log out works.

Friday — Week 3 check

- [ ] All 4 PRs open; `PROGRESS.md` updated; `npm run lint` passes (no deep imports between modules).
- [ ] The users module has a `README.md` (copy the shape of the catalog module README).

Week 4: Polish and a mini project (calendar days 22–28)

Day	Learn	Task
Monday	Online/offline state	Task 4.1 Offline banner
Tuesday	Module public APIs and import rules	Task 4.2 Home "Team" widget
Wednesday	Building a module on your own	Task 4.3 Cart tab
Thursday	Screen states and accessibility	Task 4.4 Cart summary
Friday	–	Catch-up + final check

Task 4.1 — "You're offline" banner

When: Monday (Day 16). Difficulty: medium.

Concepts you need: reading the network state · a shared component · the root layout.

What to learn first:

- expo-network
- Read `src/shared/lib/query-client.ts` (how the app already tracks online status)

The task:

1. Create `src/shared/components/offline-banner.tsx`: a thin bar (`bg-muted`, `text-muted-foreground`, `WifiOff` icon) shown only when offline. Hint: `useNetworkState()`.
2. Show it in `src/app/_layout.tsx`, above the screens.

Done when:

- [ ] Airplane mode on: the banner shows. Off: it disappears.

Task 4.2 — "Team" widget on Home

When: Tuesday (Day 17). Difficulty: medium.

Concepts you need: a module's public API · the import rules · reusing a component across modules.

What to learn first:

- ARCHITECTURE.md: "The import rules"
- Read how `src/modules/catalog/index.ts` exports `TopRatedCarousel`

The task:

1. In the users module, build a widget: 5 user avatars in a row and a "See all" button that opens the Users tab.
2. Export it from `src/modules/users/index.ts` and show it on the Home screen (`src/modules/home/features/dashboard/home-screen.tsx`), importing only from `@/modules/users`.

Done when:

- [ ] Home shows the Team row; `npm run lint` passes.

Task 4.3 — Cart tab (mini project, part 1)

When: Wednesday (Day 18). Difficulty: medium.

Concepts you need: everything from Week 3, on your own this time.

What to learn first:

- DummyJSON carts
- ADDING-A-MODULE.md (use "carts" instead of "users")

The task:

1. Create the `carts` module data layer.
2. Add a **Cart** tab showing user 1's cart (`GET /carts/user/1`): title, quantity and price for each product.

Done when:

- [ ] The Cart tab lists the products of user 1's cart.

Task 4.4 — Cart summary and screen states (mini project, part 2)

When: Thursday (Day 19). Difficulty: medium.

Concepts you need: loading / error / empty states · accessibility labels · dark mode.

What to learn first:

- React Native: accessibility
- Read `src/shared/components/screen-states.tsx`

The task:

1. Add a summary card at the bottom: total, and the discounted total in `text-success`.

2. Add loading, error and empty ("Your cart is empty") states.
3. Give every button an `accessibilityLabel`. Add a `README.md` for the carts module.

Done when:

- [ ] The Cart tab works in light and dark mode, with all states.

Friday — Final check (Day 20)

- [ ] All 16 task PRs open; `PROGRESS.md` complete; `npm run validate` passes.
- [ ] Try VoiceOver (iPhone) or TalkBack (Android) on the Users and Cart tabs and fix what's hard to use.

Calendar days 29–30: Look back

1. Record a **2-minute screen recording** of your app showing everything you added.
2. Fill in the self-check in `PROGRESS.md`.

Self-check: rate each from 1 (not at all) to 5 (confident):

- I can build a screen with the app's components and theme colors.
- I can add a route and navigate to it.
- I can load data with a schema, a service and a React Query hook.
- I can change data with a mutation and see the screen update.
- I can build a form with validation.
- I can create a new module by following the recipe.
- I can get unstuck on my own.

`PROGRESS.md` template

```
# Progress — <your name>

| Task | Name                | Day | Status (done / partly / skipped) | PR link | Hours |
Blockers |
| ---- | ------------------- | --- | -------------------------------- | ------- | ----- | ---
----- |
| 1.1  | Support row         | 1   |                                  |         |       |
|
| 1.2  | Rating stars        | 2   |                                  |         |       |
|
| 1.3  | Details card        | 3   |                                  |         |       |
|
| 1.4  | Reviews screen      | 4   |                                  |         |       |
|
| 2.1  | Category screen     | 6   |                                  |         |       |
|
| 2.2  | Sort by price       | 7   |                                  |         |       |
|
| 2.3  | Out-of-stock action | 8   |                                  |         |       |
|
| 2.4  | Discount field      | 9   |                                  |         |       |
|
| 3.1  | Users data layer    | 11  |                                  |         |       |
|
| 3.2  | Users tab           | 12  |                                  |         |       |
|
| 3.3  | User profile        | 13  |                                  |         |       |
|
| 3.4  | Login               | 14  |                                  |         |       |
|
| 4.1  | Offline banner      | 16  |                                  |         |       |
|
| 4.2  | Home Team widget    | 17  |                                  |         |       |
|
| 4.3  | Cart tab            | 18  |                                  |         |       |
|
| 4.4  | Cart summary        | 19  |                                  |         |       |
|
```

For the team lead: tracking progress

No teaching needed. Check once or twice a week:

- **PRs:** one PR per task, titled `Task 2.3: …`, on each trainee's fork. 4 per week is on track.
- **`PROGRESS.md`:** status, hours and blockers per task.

Trainee	Week 1	Week 2	Week 3	Week 4	Self-check avg	Still active?
	/4	/4	/4	/4	/5	yes / no

Good signs: a PR for every task, `npm run validate` passing, blockers written down instead of silence.

After the 20 days

Next topics, once comfortable: protected routes and token refresh, optimistic updates, component tests, performance profiling, development builds and EAS (publishing), push notifications.

Links: Expo tutorial · React Native docs · TanStack Query · NativeWind · React Native Reusables